

Text Clustering as Classification with LLMs

Chen Huang

Singapore University of Technology and Design

Singapore, Singapore

chen_huang@mymail.sutd.edu.sg

Guoxiu He*

East China Normal University

Shanghai, China

gxhe@fem.ecnu.edu.cn

Abstract

Text clustering serves as a fundamental technique for organizing and interpreting unstructured textual data, particularly in contexts where manual annotation is prohibitively costly. With the rapid advancement of Large Language Models (LLMs) and their demonstrated effectiveness across a broad spectrum of NLP tasks, an emerging body of research has begun to explore their potential in the domain of text clustering. However, existing LLM-based approaches still rely on fine-tuned embedding models and sophisticated similarity metrics, rendering them computationally intensive and necessitating domain-specific adaptation. To address these limitations, we propose a novel framework that reframes text clustering as a classification task by harnessing the in-context learning capabilities of LLMs. Our framework eliminates the need for fine-tuning embedding models or intricate clustering algorithms. It comprises two key steps: first, the LLM is prompted to generate a set of candidate labels based on the dataset and then merges semantically similar labels; second, it assigns the most appropriate label to each text sample. By leveraging the advanced natural language understanding and generalization capabilities of LLMs, the proposed approach enables effective clustering with minimal human intervention. Experimental results on diverse datasets demonstrate that our framework achieves comparable or superior performance to state-of-the-art embedding-based clustering techniques, while significantly reducing computational complexity and resource requirements. These findings underscore the transformative potential of LLMs in simplifying and enhancing text clustering tasks. We make our code available to the public for utilization¹.

CCS Concepts

• Computing methodologies → Natural language generation.

Keywords

Large Language Model, Text Clustering, Text Classification

ACM Reference Format:

Chen Huang and Guoxiu He. 2025. Text Clustering as Classification with LLMs. In *Proceedings of the 2025 Annual International ACM SIGIR Conference*

*Corresponding author.

¹<https://github.com/ECNU-Text-Computing/Text-Clustering-via-LLM>, we also provide the supplementary Appendix within the repository.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGIR-AP 2025, Xi'an, China

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-2218-9/2025/12

<https://doi.org/10.1145/3767695.3769519>

on Research and Development in Information Retrieval in the Asia Pacific Region (SIGIR-AP 2025), December 7–10, 2025, Xi'an, China. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3767695.3769519>

1 Introduction

Text clustering is a fundamental task in natural language processing (NLP), which aims to group similar texts based on their content without prior labeling. It is widely applied in scenarios where manual annotation is costly or impractical, such as improving community detection in social media [44, 58], identifying emerging topics [9, 42], analyzing large-scale textual datasets [2, 37], structuring unorganized information [13], and enhancing document retrieval [1, 3, 12]. Despite its importance, traditional text clustering faces notable challenges in both methodology and implementation.

Traditional text clustering methods typically involve transforming textual data into numerical representations and applying clustering algorithms to group similar texts based on these representations. A common approach is to use pre-trained embedding models [16, 38, 52, 57] to convert text into dense vector embeddings that capture semantic relationships. These embeddings are then clustered using algorithms like K-means [35], DBSCAN [18], or hierarchical clustering [27]. However, fine-tuning embeddings for domain-specific tasks is computationally expensive and requires labeled data [51]. Besides, clustering algorithms are sensitive to hyperparameters, such as the number of clusters and distance metrics, which often need manual tuning based on expert knowledge. The resulting clusters also lack interpretability, as clustering models typically do not produce meaningful labels for the groups. These challenges make traditional clustering approaches less flexible and efficient, particularly when applied to diverse, large-scale text datasets.

Recent advancements in LLMs, such as the GPT series [6, 40, 41], have showcased remarkable reasoning performance across a wide range of NLP tasks [24, 50, 69]. These models can comprehend and generate human-like text with great in-context learning abilities [23, 65], making them potential candidates for clustering tasks. However, existing LLM-based clustering methods [56, 59, 70] still rely on external embedding models like BERT or E5 and traditional clustering techniques such as K-means, thereby inheriting the same hyperparameter tuning and fine-tuning constraints. Moreover, API-based LLMs do not provide direct access to their internal embeddings, limiting their adaptability in clustering applications.

To address these challenges, we propose a novel two-stage LLM-driven clustering framework that reframes text clustering as a classification task, leveraging the generative and reasoning capabilities of LLMs. Our approach consists of two key stages: label generation stage and text classification stage. The LLM processes input texts sequentially in mini-batches and generates meaningful labels based on content similarities. This dynamic label generation enables adaptive clustering without requiring predefined cluster

numbers or embedding fine-tuning. Once labels are established, the LLM classifies the remaining texts according to the generated labels, effectively grouping similar texts without the need for conventional clustering algorithms. By transforming clustering into a classification task, our approach effectively addresses the core limitations of traditional clustering methods. First, it eliminates the need for fine-tuned embeddings, making the method highly adaptable across different datasets without requiring extensive model customization. Second, it avoids the necessity of manual hyperparameter tuning, thereby reducing reliance on expert knowledge and mitigating the risks associated with suboptimal parameter selection. Third, our framework enhances interpretability by leveraging the LLM’s capability to generate human-readable labels, providing meaningful insights into the resulting clusters. Lastly, by processing data in sequential mini-batches, it overcomes the input length limitations of LLMs, enabling efficient clustering of large-scale text datasets without compromising performance.

We evaluate our framework on five datasets across diverse NLP tasks, including topic mining, emotion detection, intent discovery, and domain classification, with cluster granularities ranging from 18 to 102. Our results demonstrate that the proposed approach achieves comparable or superior clustering performance compared to state-of-the-art methods while significantly reducing computational overhead and manual effort.

Our key contributions are summarized as follows:

- We propose a novel LLM-driven framework that reframes clustering as a classification task, leveraging the label generation and reasoning capabilities of LLMs. Compared to recent LLM-based clustering methods, this automated clustering pipeline eliminates the need for fine-tuned embeddings and hyperparameter tuning.
- By utilizing LLMs’ powerful summarization and classification capabilities, our framework generates high-quality, human-understandable labels, providing a practical and scalable alternative to traditional clustering approaches.
- Extensive experiments on five diverse datasets demonstrate that our method achieves state-of-the-art results compare to recent LLM-based clustering methods, while being more computationally efficient and adaptable to various domains.

2 Related Work

2.1 Clustering

Clustering as a fundamental task in machine learning, has been applied to various data types, including texts [2, 4, 36, 61], images [11, 33, 43, 47, 60, 64], and graphs [26, 28, 45, 49, 54, 66, 72]. Traditional clustering methods, such as K-means [35] and DBSCAN [18], have been widely applied in text clustering due to their simplicity and efficiency. K-means is an iterative partitioning algorithm that assigns data points to clusters based on their distance from centroids, requiring the number of clusters to be predefined. This limitation makes it less adaptable when the true number of clusters is unknown. DBSCAN, on the other hand, is a density-based clustering method that identifies clusters of arbitrary shape and does not require a predefined cluster number. However, it struggles with high-dimensional data, such as text embeddings, and requires careful tuning of distance thresholds. Both methods rely heavily on well-crafted feature representations, and their performance is

sensitive to the choice of similarity measures and hyperparameters. Additionally, as they do not provide meaningful cluster labels, making it difficult to analyze the structure of clustered text groups.

In addition to research aimed at improving traditional machine learning algorithms for clustering [17, 19, 34], recent studies have increasingly focused on leveraging deep neural networks, which model instance similarities by learning meaningful representations [5, 22, 25, 31, 46, 48, 71]. For example, Yang et al. [63] propose a recurrent network for joint unsupervised learning of deep representations in clustering. Caron et al. [8] jointly learn the parameters of neural networks and the cluster assignments of the resulting features. Tao et al. [53] combine instance discrimination and feature decorrelation to present a clustering-friendly representation learning method. While these methods have demonstrated strong performance, they require an additional training process to obtain feature representations, followed by the application of traditional clustering algorithms [21].

The reliance of these approaches on extensive training constrains their adaptability across datasets, as models must be retrained for each new domain, leading to significant computational costs. In contrast, our framework of transferring text clustering into classification task eliminates the need for fine-tuning or embedding-specific training, enabling seamless adaptation to diverse datasets without incurring additional computational overhead.

2.2 Adding Explanations to Text Clusters

While previous clustering algorithms do not necessarily produce interpretable clusters [10], studies pay attention to explaining the clusters with semantically meaningful expressions [62, 68]. Treeratpituk and Callan [55] assign labels to hierarchical clusters and assesses potential labels by utilizing information from the cluster itself, its parent cluster, and corpus statistics; Carmel et al. [7] propose a framework that selects candidate labels from external resources like Wikipedia to represent the content of the cluster; Navigli and Crisafulli [39] induce word senses when clustering the result based on their semantic similarity; Zhang et al. [67] iteratively identify general terms and refines the sub-topics during clustering to split coarse topics into fine-grained ones. However, label or phrase level added information is limited in describing a complex cluster [59], and labels assigned may have similar meanings, resulting in overlapping labels. Thus, more in-depth expressions and better granularity control are required to make clusters more explainable and accurate.

By utilizing LLMs to generate interpretable cluster labels, our method enhances the explainability of clustering results, providing meaningful insights into the grouped data. This formulation not only improves clustering quality but also significantly reduces the complexity of the clustering process.

2.3 Text Clustering using LLMs

Recent rapid development of Large Language Models (LLMs), such as GPT series [6, 40, 41], has demonstrated the powerful comprehensive language capability of LLMs and some works has been using LLMs in text clustering task. Wang et al. [59] utilize LLMs to propose explanations for the cluster and classify the samples based on the generated explanations; De Raedt et al. [14] collect descriptive

utterance labels from LLMs with well-chosen prototypical utterances to bootstrap in-context learning; Kwon et al. [30] use LLMs to label the description of input data and cluster the labels with given K . Besides explanation and label generation, Viswanathan et al. [56] expand documents' keyphrases, generate pairwise constraints and correct low-confidence points in the clusters via LLMs, Zhang et al. [70] leverage feedbacks from LLMs to improve smaller embedders, such as Instructor [52] and E5 [57], and prompt LLMs for helps on clustering granularity.

All these methods use LLMs in an indirect way that LLMs only process part of the input data and do not see the whole dataset. In contrast, our proposed framework dynamically generates cluster labels and assigns data points in a sequential manner, mitigating the challenges of high-dimensional text clustering and leveraging the full advantages of generative and reasoning capabilities of LLMs.

3 Methodology

In this work, we propose a two-stage framework that utilizes a single LLM for text clustering tasks. To better leverage the generative and classification capabilities of LLMs, we transform the clustering task into a label-based classification task, allowing the LLM to process the data more effectively. As illustrated in Figure 1 and summarized in Algorithm 1, unlike previous text clustering methods such as ClusterLLM [70] that calculate distances between data points in vector space, our framework does not require fine-tuning for better representation or a pre-assigned cluster number K . We first prompt the LLM to generate potential labels for the data. After merging similar labels, we then prompt the LLM to classify the input data based on these generated labels. The detailed steps of our framework are introduced in the following sections.

3.1 Task Definition

For text clustering, given an unlabeled dataset $\mathcal{D} = \{d_i\}_{i=1}^N$, where N is the size of the corpus, the goal is to output K subsets of $\mathcal{D}$ as $\mathcal{C} = \{c_j\}_{j=1}^K$, where K represents the number of clusters and each c_j represents a cluster, such that $d_1 \in c_j$ and $d_2 \in c_j$ if d_1 and d_2 belong to the same cluster. We transform text clustering task into classification task in this work. Specifically, given the dataset $\mathcal{D}$, the model first generates a set of labels $\mathcal{L} = \{l_k\}_{k=1}^{K'}$ based on the content of the dataset, where K' is the number of labels. Subsequently, each data $d_i \in \mathcal{D}$ will be classified into one of the labels $l \in \mathcal{L}$ and the input dataset will be clustered into K' clusters $\mathcal{C}' = \{c'_j\}_{j=1}^{K'}$.

3.2 Label Generation Using LLMs

In this section, we explore the process of forming a label-generation task to obtain potential labels for clusters using LLMs. Given the few-shot capabilities of LLMs [6], we will provide several example label names to fully utilize the in-context learning ability of LLMs.

3.2.1 Potential Label Generation. Since inputting an entire dataset into LLMs is impractical due to context length limitations, we input the dataset in mini-batches and then aggregate the potential labels. Subsequently, we prompt the model to merge similar labels to adjust the granularity of the clusters. Specifically, given a batch size B , we will first prompt the LLM with B instances along with n example

Algorithm 1 LLM-based Text Clustering as Classification

Require: Unlabeled dataset $D = \{d_1, d_2, \dots, d_N\}$, batch size B , few-shot labels L_{few}

Ensure: Clusters $\mathcal{C}' = \{c'_1, c'_2, \dots, c'_{K'}\}$

```

1: Initialize  $L_{\text{all}} \leftarrow \emptyset$ 
2: Split  $D$  into  $\lceil N/B \rceil$  mini-batches:  $\{D_1, D_2, \dots, D_M\}$ 
3: for each batch  $D_b \in \{D_1, D_2, \dots, D_M\}$  do
4:   Prompt LLM with  $(D_b, L_{\text{few}})$  using  $P_g$  to generate potential labels  $L_b$ 
5:    $L_{\text{all}} \leftarrow L_{\text{all}} \cup L_b$ 
6: end for
7:  $L_{\text{unique}} \leftarrow \text{unique}(L_{\text{all}})$ 
8:  $L_{\text{final}} \leftarrow$  Prompt LLM with  $L_{\text{unique}}$  using  $P_m$  to merge similar labels
9: Initialize  $\mathcal{C}' \leftarrow \emptyset$ 
10: for each text  $d \in D$  do
11:    $c_d \leftarrow$  Prompt LLM with  $(d, L_{\text{final}})$  using  $P_a$  to assign a label
12:   Add  $d$  to cluster  $c_d$  in  $\mathcal{C}'$ 
13: end for
14: return  $\mathcal{C}'$ 
```

label names to generate potential labels for the input data using a prompt P_g , where the dataset is divided into $\frac{N}{B}$ mini-batches for processing:

$$\mathcal{L}' = P_g(I_{\text{generate}}, \mathcal{D}', l) \quad (1)$$

where I_{generate} is the label generation task instruction, $\mathcal{D}' = \{d_i\}_{i=1}^B$ is the input data in mini-batches of the size B , and l represents the n given label names.

3.2.2 Potential Labels Aggregation and Mergence. After obtaining all the potential labels from LLMs, we aggregate the labels generated from each mini-batch together:

$$\mathcal{L}_{\text{unique}} = \{l \mid l \in \mathcal{L}'\} \quad (2)$$

To avoid redundant duplication of final clusters caused by the LLM producing different descriptions for the same label, we further prompt the LLM to merge labels with similar expressions:

$$\mathcal{L} = P_m(I_{\text{merge}}, \mathcal{L}_{\text{unique}}) \quad (3)$$

where I_{merge} is the instructions of the merging task.

3.3 Given Label Classification

Given the potential labels for the entire dataset, we can now obtain the final clusters by performing label classification using LLMs. For each input instance, we prompt the LLM to assign a label from the previously generated potential labels:

$$c'_j = P_a(I_{\text{assign}}, d_j, \mathcal{L}) \quad (4)$$

where c'_j is the cluster that the LLM classifies d_j into and I_{assign} is the instruction of the assigning task. After assigning all the data in the dataset according to the labels, we finally get the text clustering result $\mathcal{C}' = \{c'_j\}_{j=1}^{K'}$. For the detailed prompt template and instructions I_{generate} , I_{merge} , and I_{assign} , please refer to the Appendix.

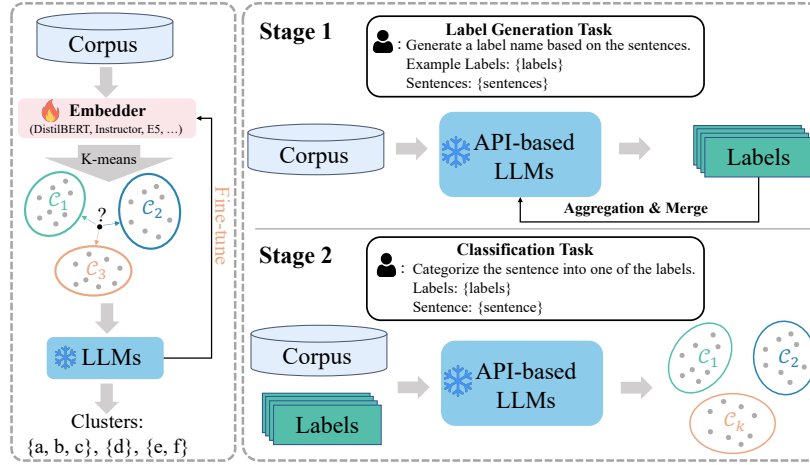


Figure 1: A comparison between other methods using LLMs (left) and our framework (right) for text clustering. Our framework transforms the clustering task into a text classification task by generating potential labels (Stage 1) and classifying input sentences according to the labels (Stage 2) using LLMs.

Table 1: Dataset statistics. #clusters denotes the number of true label clusters, while #data represents the number of instances within each cluster.

Task	Name	#clusters	#data
Topic	ArxivS2S	93	3674
Emotion	GoEmo	27	5940
Domain	Massive-D	18	2974
Intent	Massive-I	59	2974
	MTOP-I	102	4386

4 Experiment

4.1 Dataset Description

We extensively evaluate our framework on five datasets encompassing diverse tasks, including topic mining, emotion detection, intent discovery, and domain discovery. Each dataset has different granularities, ranging from 18 to 102 clusters.

ArxivS2S [38] is a text clustering dataset in the domain of academic, it contains sentences describing a certain domain. **GoEmo** [15] is a fine-grained dataset for emotion detection, multi-label or neutral instances are removed for text clustering purpose. **Massive-I/D** [20] and **MTOP-I** [32] are datasets originally used for classification but adapted for text clustering. “I” denotes intent and “D” denotes domain. Following Zhang et al. [70], all the datasets are splitted into large- and small-scale versions with the same number of clusters. Dataset statistics summary is shown in Table 1. We use small-scale version of datasets to reduce cost.

4.2 Implementation Details

We use GPT-3.5-turbo as the query LLM for label generation and given label classification. Responses are controlled by adding a postfix: “Please response in JSON format”. Detailed prompts and instructions are provided in Appendix A. We then extract the labels from the JSON response. During label generation, label names are provided to the LLM as examples. We set the number of given label names to 20% of the total number of labels in the dataset. To account

for context length limitations, we set the mini-batch size B to 15, meaning the LLM receives 15 input sentences at a time to generate potential labels.

4.3 Evaluation Metrics

Following [14] and [70], we evaluate clustering quality using three metrics: Accuracy (ACC), Normalized Mutual Information (NMI), and Adjusted Rand Index (ARI). Accuracy measures how well the predicted clusters align with the true labels, which requires addressing the inherent lack of ordering in clustering labels. To solve this, the Hungarian algorithm [29] is used to find an optimal mapping between predicted and true labels. Once aligned, ACC is calculated as the proportion of correctly assigned labels. NMI, on the other hand, quantifies the similarity between the true and predicted clusters by using mutual information to measure how much information about the true labels can be gained from the predicted clusters. This is then normalized by the average entropy of the two label sets, making NMI robust to differences in cluster sizes and independent of whether true or predicted clusters are treated as the ground truth. Lastly, ARI evaluates clustering quality by comparing pairs of samples, extending the Rand Index by accounting for the agreement expected by random chance. This adjustment ensures that ARI values close to zero indicate clustering performance no better than random, with negative values suggesting worse-than-random clustering and positive values reflecting better clustering.

4.4 Compared Baselines

To demonstrate that our framework of using LLMs directly without embedding or fine-tuning can improve the text clustering results, other than traditional clustering algorithm, we compare our result with methods that also utilize LLMs to different extents. Since different models all evaluated on different datasets, to better compare the performance of baseline models and our framework, we implement the baseline models on the five datasets using the source code provided by the authors.

Table 2: Experiment results of text clustering on five datasets, evaluated using Accuracy, NMI, and ARI. Best results are highlighted in bold. *LLM_known_labels* represents the theoretical upper bound for LLMs in this task. * indicates significant improvement under statistical significance tests with $p < 0.05$.

Methods	ArxivS2S			GoEmo			Massive-I			Massive-D			MTOP-I		
	ACC	NMI	ARI	ACC	NMI	ARI	ACC	NMI	ARI	ACC	NMI	ARI	ACC	NMI	ARI
K-means (E5)	31.21	54.47	17.01	22.14	21.26	9.64	52.79	70.76	39.03	62.21	65.42	47.69	34.48	71.47	26.35
K-means (Instructor)	25.11	48.48	12.39	25.19	21.54	17.03	56.55	74.49	42.88	60.41	67.31	43.90	33.04	71.46	26.72
DBSCAN (E5)	24.31	38.67	15.60	16.52	18.63	11.2	51.80	72.38	35.54	57.58	60.58	41.96	37.84	71.49	23.81
DBSCAN (Instructor)	26.17	42.56	15.57	18.16	20.32	15.74	48.69	67.81	34.26	55.39	60.97	38.35	35.64	69.63	26.45
IDAS	16.79	41.56	6.68	15.24	12.00	5.43	51.33	68.38	38.29	54.65	57.32	42.49	33.91	68.70	27.90
PAS	36.50	16.37	18.15	11.34	2.84	10.14	19.62	28.99	9.56	40.63	30.99	22.80	50.88	64.88	41.83
Keyphrase Clustering	23.48	45.57	10.68	20.19	18.73	12.36	55.42	74.38	41.26	54.84	63.75	37.58	30.02	72.45	28.75
ClusterLLM	26.34	50.45	13.65	26.75	23.89	17.76	60.69	77.64	46.15	60.85	68.67	45.07	35.04	73.83	29.04
Ours	38.78*	57.43*	20.55*	31.66*	27.39*	13.50	71.75*	78.00*	56.86*	64.12*	65.44	48.92*	72.18*	78.78*	71.93*
<i>LLM_known_labels</i>	41.50	57.59	20.67	38.97	28.85	18.94	75.25	78.19	58.01	69.77	69.27	55.26	73.25	80.88	73.93

K-means/DBSCAN. We use embeddings extracted from E5-large [57] and Instructor-large [52] and apply K-means/DBSCAN algorithm to obtain the text clustering result. We run the clustering five times with different seeds and calculate the average result as the final result.

IDAS² [14] identifies prototypes that represent the latent intents and independently summarizes them into labels using LLMs. Then, it encodes the concatenation of sentences and summaries for clustering. We first generate labels using GPT-3 (text-davinci-003) [6] for the five datasets used in this paper. For each test set, five JSON files are generated with different sample order, with the nearest neighbors $topk = 8$. After that, we produce the result with the generated labels and calculate the evaluation metrics.

PAS³ [59] develops a three-stage algorithm Propose-Assign-Select by prompting LLMs to generate goal-related explanations, determine whether each explanation supports each sample, and use integer linear programming to select clusters such that each sample belongs to one single cluster. We use the same experiment settings as [59] and use GPT-3.5-turbo as the proposers and google/flan-t5-xl⁴ as the assigners. For *cluster_num* parameter, we set it as the number of labels in the datasets.

Keyphrase Clustering is the best performing clustering model proposed by Viswanathan et al. [56], which expands the expression by generating keyphrases using LLM.

ClusterLLM⁵ [70] prompts LLM for insights on similar data points and fine-tunes small embedders using the LLM’s choice. It also uses GPT-3.5 to guide the clustering granularity by determining whether two data points belong to the same category. Since ClusterLLM does not present its results in the ARI metric, we also reproduce its results on the five datasets. We choose the best performing model *ClusterLLM-I-iter* for comparison. This model adopts Instructor⁶ as the embedder and applies the framework twice in an iterative way by using the previously fine-tuned model as initialization. The LLM used for triplet sampling and pairwise hierarchical sampling

is GPT-3.5-turbo. We also re-perform the framework on ArxivS2S and GoEmo datasets to obtain the *#clusters* result in granularity analysis in Section 5.2, which is not presented in the original paper. The *#clusters* result of dataset Massive-I, Massive-D, and MTOP-I is taken directly from the paper [70].

Additionally, we apply our framework with gold labels given, which performs label classification using the dataset’s ground truth cluster labels, shown as **LLM_known_labels**. This model represents the upper bound of the LLM’s performance.

5 Results

5.1 Text Clustering Results

We present the text clustering results in Table 2 and draw several key observations from the experimental findings.

Firstly, our proposed framework consistently outperforms baseline approaches across all datasets, with very few exceptions. For example, our framework achieves a significant accuracy improvement of 12.44% on the ArxivS2S dataset and even doubles the performance on MTOP-I. These results highlight the robustness and effectiveness of leveraging LLMs exclusively for text clustering tasks.

Furthermore, the observed improvements across three distinct evaluation metrics demonstrate that our framework enhances text clustering performance from multiple dimensions. This indicates that our framework not only excels at accurately identifying and differentiating distinct categories but also effectively captures the intrinsic relationships and underlying structures within the data. The consistent enhancement across metrics underscores the comprehensive impact of our framework on clustering quality.

In addition, the performance of our framework is remarkably close to the theoretical upper bound (*LLM_known_labels*), which uses ground truth cluster labels for classification. Achieving near-upper-bound performance without access to true labels demonstrates the capability of our framework to generate meaningful potential labels and refine cluster granularity through effective label merging. This underscores the ability of our framework to balance interpretability and accuracy, providing a practical and scalable alternative to traditional clustering techniques.

²<https://github.com/maarten-deraedt/IDAS-intent-discovery-with-abstract-summarization>.

³<https://github.com/ZihanWangKi/GoalEx>.

⁴<https://huggingface.co/google/flan-t5-xl>.

⁵<https://github.com/zhang-yu-wei/ClusterLLM>

⁶<https://huggingface.co/hkunlp/instructor-large>

Table 3: Granularity analysis. The results are presented in the format of “[#clusters](difference)”, where a positive difference means the model generate more #clusters than ground truth and vice versa.

Method	ArxivS2S	GoEmo	Massive-I	Massive-D	MTOP-I
GT #clusters	93	27	59	18	102
ClusterLLM	16 (-77)	56 (+29)	43 (-16)	90 (+72)	43 (-59)
Ours	122 (+29)	52 (+25)	71 (+12)	24 (+6)	83 (-19)

Overall, these results validate the strength of our framework in addressing the challenges of unsupervised text clustering, emphasizing its potential for broader applications and its reliability across diverse datasets. Our findings also highlight the significant role that LLMs can play in simplifying and improving text clustering by fully utilizing their in-context learning and generalization capabilities.

5.2 Granularity Analysis

To assess the granularity of the output clusters, we compare the final cluster number generated by our framework with those produced by ClusterLLM.

We first compare our framework’s cluster granularity with those produced by ClusterLLM to justify the effectiveness of label merging task in our framework. The smaller absolute difference in Table 3 demonstrates that our framework yields cluster counts that are more closely aligned with the true number of clusters. This improved alignment with the real cluster distribution underscores the effectiveness of our framework in better capturing the underlying data structure by merging labels with similar semantic meanings. As a result, this enhances cluster coherence and validity.

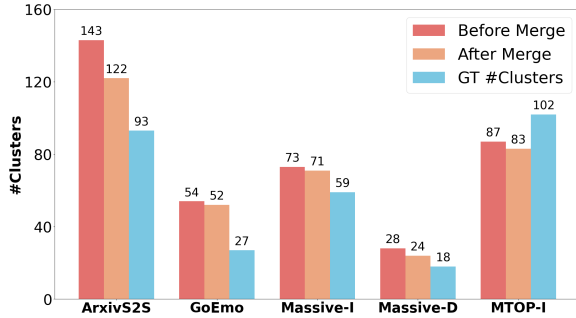


Figure 2: Label merging granularity on five datasets. “GT #Clusters” means the ground truth number of clusters in the dataset.

To demonstrate how our proposed framework handle ambiguous or overlapping categories during clustering process, we also conduct an comparative analysis on granularity before and after the merging task. Figure 2 shows that merging similar labels helps the model aggregate labels with same meanings, resulting in a cluster number closer to the the ground truth clusters. This merging method is especially effective when the number of labels is larger. For example, it aggregates 21 similar labels in the ArxivS2S dataset. Since the number of clusters can heavily impact the final clustering result, this method of improving the granularity is necessary.

This closer alignment with the actual cluster distribution highlights our framework’s ability in more accurately capturing the underlying structure of the data through merging labels that have similar semantic meanings. Consequently, this leads to improved cluster coherence and validity. The ablation test regarding label merging task in Figure 2 supports this conclusion. It compares the cluster granularity before and after the merging task and shows that performing label merging task can help the model aggregate similar labels and output a cluster number that is closer to the ground truth.

5.3 Prompt Variation Analysis

Prompt quality plays a critical role in guiding LLMs to perform effectively in downstream tasks. To assess the impact of prompt phrasing, we conduct experiments using various formulations for prompts during both the label generation and text classification stages. The results of these experiments, summarized in Table 4, reveal that while changes in prompt wording lead to minor performance fluctuations, these variations are not significant enough to alter the overall outcomes. Importantly, the performance of our proposed framework remains robust across different prompt expressions, consistently outperforming the current state-of-the-art model, ClusterLLM, in most scenarios. This consistent superiority highlights not only the effectiveness of our framework but also its adaptability to varying prompt structures. The ability to maintain strong performance despite changes in prompts underscores the generalizability of our framework, making it a reliable approach for diverse text clustering tasks. These findings reinforce the importance of prompt engineering while also demonstrating that our framework reduces dependence on precise prompt tuning, a common challenge in deploying LLMs for real-world applications.

5.4 Few-shot Label Generation

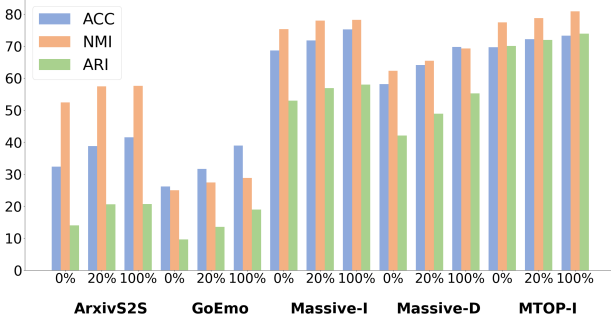
We provide the LLM with few-shot examples in the label generation task to better exploit its in-context learning capability. By observing a small number of gold labels, the LLM can infer the underlying label semantics and generate more coherent and meaningful candidate labels. To quantify this effect, we conduct experiments with varying percentages of gold labels, as illustrated in Figure 3. The 100% case represents a theoretical upper bound (“LLM_known_labels” in Section 4.4), where all true labels are given, and the LLM performs direct classification. We find that even a small number of examples (e.g., 10–15%) consistently boosts clustering performance across all metrics and datasets. This result highlights the importance of carefully designed few-shot prompts and validates our framework’s strategy of leveraging example labels in the label generation step (Section 3.2.1). Importantly, even under the 0% setting where the LLM receives no few-shot examples, it still outperforms baseline methods on most datasets. This fully unsupervised scenario highlights the strong intrinsic capability of LLMs to generate meaningful candidate labels and perform classification without external supervision.

5.5 Hyperparameter Sensitivity Analysis

We conduct extensive experiments to evaluate the influence of key hyperparameters on our framework’s performance, with a

Table 4: Prompt Variation Results. “Prompt1” and “Prompt2” refers to different prompt variation compared to original prompt used in our framework.

Methods	ArxivS2S			GoEmo			Massive-I			Massive-D			MTOP-I		
	ACC	NMI	ARI	ACC	NMI	ARI	ACC	NMI	ARI	ACC	NMI	ARI	ACC	NMI	ARI
ClusterLLM	26.34	50.45	13.65	26.75	23.89	17.76	60.69	77.64	46.15	60.85	68.67	45.07	35.04	73.83	29.04
Prompt1	36.71	51.25	17.62	29.64	26.90	13.05	70.48	73.35	55.34	60.03	59.73	42.98	72.69	72.24	68.56
Prompt2	35.96	55.97	18.56	30.94	25.02	11.55	69.09	75.98	54.44	63.94	64.59	48.14	71.41	71.69	70.82
Original Prompt	38.78	57.43	20.55	31.66	27.39	13.50	71.75	78.00	56.86	64.12	65.44	48.92	72.18	78.78	71.93

**Figure 3: ACC, NMI, ARI of our framework on five dataset with different percentage of given labels. 0% means no label is provided to the LLM, 20% means we give 20% of the total gold labels to the LLM during label generation and 100% means LLM is provided with all true labels and directly performs classification.**

particular focus on the batch size B and the percentage of provided labels used for in-context learning. In addition to the default batch size of 15, we investigate the impact of smaller and larger batches by testing values of 10 and 20. As reported in Table 5, the results demonstrate that varying the batch size does not significantly affect the overall performance trend of our framework. This observation indicates that our approach is relatively robust to changes in batch-level granularity, suggesting that the LLM’s reasoning ability is not heavily dependent on the exact number of samples seen in each batch.

One possible reason for this robustness is that the core task — generating and merging meaningful labels — is driven by semantic understanding rather than batch statistics. Since each batch is processed independently by the LLM with self-contained prompts, altering B mainly influences computational efficiency rather than the quality of label generation or classification. Larger batch sizes (e.g., $B = 20$) slightly increase the context size but do not introduce additional semantic information, while smaller batches (e.g., $B = 10$) simply reduce the number of examples the model observes in one pass without degrading performance. Consequently, the choice of B can be guided more by resource and latency considerations rather than accuracy concerns, making our framework more flexible in real-world applications.

Moreover, we explore the effect of varying the proportion of provided labels on performance by testing three different settings: 10%, 15%, and 25%. Table 6 shows that our framework benefits notably from the presence of a moderate number of given labels,

leveraging the in-context learning capability of LLMs. When the number of provided label examples is small (e.g., 10%), the performance improves steadily as more few-shot examples are included (e.g., 15%). However, when an excessive number of examples are introduced (25%), the model’s performance slightly deteriorates. We hypothesize that this decline is due to prompt overload, where too many examples cause the model to lose focus on the underlying clustering logic or overfit to the examples rather than generalizing across the dataset. These findings emphasize the importance of carefully balancing the number of in-context examples to achieve optimal performance.

5.6 Cost Comparison

We report a comparison of the monetary cost and wall-clock time between our proposed API-based method and a fine-tuning-based approach in Table 7.

Let N denote the size of the evaluation dataset, and let D be the size of the training set with sequence length L , trained for E epochs. The input and output token lengths for the LLM are represented by T_{in} and T_{out} , respectively. Since API calls can be executed in parallel, we assume a parallel throughput of R requests per second.

The standard API price for GPT-3.5-turbo is \$0.50 per million input tokens and \$1.50 per million output tokens⁷. For fine-tuning, we use the average rental cost of \$1.73 per A100 (80GB) GPU per hour across providers⁸. Following Zhang et al. [70], we set $D = 60,000$, $E = 15$, and $L = 512$. We fix $T_{\text{in}} = 200$, $T_{\text{out}} = 50$, and $R = 100$ for all calculations.

Throughput estimation. We estimate throughput for fine-tuning and inference based on the effective processing rate of transformer encoders on 4×A100 GPUs (80GB). The training throughput is

$$\tau_{\text{train}} = \frac{B \cdot G}{t_{\text{iter}}}, \quad \tau_{\text{inf}} = \frac{B_{\text{inf}} \cdot G}{t_{\text{iter}}^{\text{inf}}}, \quad (5)$$

where B is the per-GPU training batch size, B_{inf} is the per-GPU inference batch size, G is the number of GPUs, t_{iter} is the iteration time during training, and $t_{\text{iter}}^{\text{inf}}$ is the iteration time during forward-only inference. In practice, with $B = 64$, $G = 4$, and $t_{\text{iter}} \approx 0.85\text{s}$ for sequence length $L = 512$, we obtain $\tau_{\text{train}} \approx 64 \times 4 / 0.85 \approx 300$ samples/s. For inference, with $B_{\text{inf}} = 128$ and $t_{\text{iter}}^{\text{inf}} \approx 0.5\text{s}$, we obtain $\tau_{\text{inf}} \approx 128 \times 4 / 0.5 \approx 1000$ requests/s. These values are consistent with reported throughput benchmarks for BERT-style encoders on

⁷<https://platform.openai.com/docs/pricing>

⁸Sources: <https://lambda.ai/service/gpu-cloud>, <https://www.runpod.io/gpu-models/a100-pcie>

Table 5: Experiments on different batch size B . We use batch size of 15 in our presented method.

Batch Size	ArxivS2S			GoEmo			Massive-I			Massive-D			MTOP-I		
	ACC	NMI	ARI	ACC	NMI	ARI	ACC	NMI	ARI	ACC	NMI	ARI	ACC	NMI	ARI
10	37.66	52.19	18.38	33.85	24.62	15.60	69.76	65.10	48.87	55.91	52.39	38.23	68.29	69.99	70.54
15	38.78	57.43	20.55	31.66	27.39	13.50	71.75	78.00	56.86	64.12	65.44	48.92	72.18	78.78	71.93
20	35.62	53.88	18.71	28.25	25.89	14.45	71.18	77.47	51.26	61.04	62.90	46.27	70.82	70.28	66.52

Table 6: Experiments on different given label percentage. We use 20% in our presented method.

Given n labels	ArxivS2S			GoEmo			Massive-I			Massive-D			MTOP-I		
	ACC	NMI	ARI	ACC	NMI	ARI	ACC	NMI	ARI	ACC	NMI	ARI	ACC	NMI	ARI
10%	28.80	44.16	15.98	26.28	17.28	11.32	60.20	67.54	49.09	54.54	58.12	43.05	54.53	68.34	46.61
15%	33.55	49.51	16.97	28.02	22.23	11.51	64.31	72.42	51.25	63.87	62.45	48.36	63.10	72.43	62.44
20%	38.78	57.43	20.55	31.66	27.39	13.50	71.75	78.00	56.86	64.12	65.44	48.92	72.18	78.78	71.93
25%	36.57	56.45	19.86	32.39	29.36	13.54	69.99	78.24	55.34	65.64	61.72	45.53	65.85	70.46	61.48

Table 7: Cost/time comparison between API-based method (API) and fine-tuning-based approach (FT). N represents the size of evaluation data. All times are reported in minutes.

N	API Cost	API Time	FT Cost	FT Time
20k	\$3.50	3.3	\$9.65	83.6
100k	\$17.50	16.7	\$9.80	85.0
500k	\$87.50	83.3	\$10.57	91.6

A800 GPUs and represent conservative but realistic estimates that balance compute, memory, and data loader efficiency.

API-based method. The API cost and time are given by

$$\text{Cost}_{\text{API}} = \frac{N}{10^6} (T_{\text{in}} \cdot 0.50 + T_{\text{out}} \cdot 1.50), \quad t_{\text{API}} = \frac{N}{R}. \quad (6)$$

Note that the batch size does not affect the total token usage, and therefore does not appear in the formula.

Fine-tuning-based method. The training time and cost are

$$t_{\text{train}} = \frac{D \cdot E}{\tau_{\text{train}}}, \quad \text{Cost}_{\text{train}} = \frac{t_{\text{train}}}{3600} \cdot 4 \cdot C_{\text{GPU}}, \quad (7)$$

where C_{GPU} is the hourly rental price per GPU. Inference requires

$$t_{\text{inf}} = \frac{N}{\tau_{\text{inf}}}, \quad \text{Cost}_{\text{inf}} = \frac{t_{\text{inf}}}{3600} \cdot 4 \cdot C_{\text{GPU}}. \quad (8)$$

The total cost and time for the fine-tuned model are therefore

$$\text{Cost}_{\text{FT}} = \text{Cost}_{\text{train}} + \text{Cost}_{\text{inf}}, \quad t_{\text{FT}} = t_{\text{train}} + t_{\text{inf}}. \quad (9)$$

From the results, we observe that our API-based method scales linearly with the evaluation size N and requires no setup cost, making it attractive for quick, small-scale experiments. In contrast, the fine-tuning approach incurs a one-time training overhead proportional to the size of the training data, but after training, its inference cost grows linearly with N at a much smaller constant factor due to high GPU throughput. Consequently, for small N , the API-based method is cheaper and faster to deploy. At moderate N ,

the API-based approach is faster in wall-clock time but incurs a higher cost than fine-tuning. However, as N increases further, the fine-tuned approach becomes more cost-effective, since the upfront training expense is amortized and inference can be carried out at low marginal cost.

6 Conclusion

We propose a novel approach to text clustering that leverages LLMs exclusively, eliminating the need for additional embedding models or traditional clustering algorithms. Our two-stage framework reframes the text clustering problem as a combination of label generation and classification tasks, enhancing both the performance and interpretability of clustering results by assigning meaningful, human-readable labels to the clusters. In the first stage, we prompt LLMs with batches of input sentences to generate explainable and contextually relevant potential labels. To ensure consistency and clarity, similar labels are merged into a unified set of candidate cluster labels. In the second stage, we classify each input sentence into one of these refined labels using the LLM, effectively completing the clustering process. This framework capitalizes on the advanced natural language understanding, generation, and classification capabilities of LLMs without requiring any additional fine-tuning or task-specific training. The comprehensive knowledge encoded in LLMs from pre-training on diverse and extensive datasets significantly enhances our framework’s domain adaptability, making it well-suited for clustering tasks across various applications and industries.

Extensive experimental results demonstrate the effectiveness of our framework, showcasing superior clustering performance and improved granularity compared to state-of-the-art methods. Furthermore, this framework’s simplicity and adaptability make it a promising solution for applications requiring scalable and interpretable text clustering. In the future, we aim to explore more cost-efficient strategies and finer-grained clustering methods, leveraging the evolving capabilities of LLMs to further enhance performance and reduce resource consumption.

Table 8: Prompt template and instructions used in this paper. In this template, words inside {} should be replaced by corresponding variables during experiments.

Task	Prompt template with instruction $\mathcal{I}$
Label Generation $\mathcal{P}_g$	Given the labels, under a text classification scenario, can all these text match the label given? If the sentence does not match any of the label, please generate a meaningful new label name. Labels: {given_labels} Sentences: {sentence_list} You should NOT return meaningless label names such as ‘new_label_1’ or ‘unknown_topic_1’ and only return the new label names, please return in json format like: {json_example}
Aggregating and merging labels $\mathcal{P}_m$	Please analyze the provided list of labels to identify entries that are similar or duplicate, considering synonyms, variations in phrasing, and closely related terms that essentially refer to the same concept. Your task is to merge these similar entries into a single representative label for each unique concept identified. The goal is to simplify the list by reducing redundancies without organizing it into subcategories or altering its fundamental structure. Here is the list of labels for analysis and simplification:{label_list}. Produce the final, simplified list in a flat, JSON-formatted structure without any substructures or hierarchical categorization like: {json_example}
Given label classification $\mathcal{P}_a$	Given the label list and the sentence, please categorize the sentence into one of the labels. Label list: {label_list} Sentence: {sentence} You should only return the label name, please return in json format like: {json_example}

7 Limitation

Our work has limitations in the following senses. First, as our work relies exclusively on LLMs for text clustering and does not fine-tune smaller embedders for better representation, more processing is required through LLMs. This results in increased API usage and higher associated costs. Since we use the LLM for given-label classification, the number of API calls is proportional to the dataset size. While the savings in computational costs can offset a significant portion of this API cost increase, this remains a cost limitation when dealing with large datasets. Second, while our framework achieves better granularity in clustering results compared to other LLM-based methods like ClusterLLM, it still lacks fine-grain control. Third, during the label generation process, without explicit guidelines or standardization protocols, LLMs might produce labels that vary widely in phrasing and granularity. To manage this, we apply a merging process using the LLM to control the granularity of the labels generated. However, LLMs might not consistently merge synonymous labels or accurately distinguish between polysemous words, leading to fragmented clusters. Additionally, labels or words with multiple meanings could result in ambiguous labeling. This issue can be mitigated by adding explicit explanations for the generated labels.

8 Future Work

In future work, we aim to enhance our framework by incorporating user feedback to improve label accuracy and granularity, leveraging human expertise and knowledge. By allowing users to provide feedback on generated labels, the LLM can refine label quality and better manage granularity. Since our framework is built on LLMs, this interaction can be efficiently facilitated through text, making it accessible to users without algorithmic expertise. This approach is particularly advantageous in real-world scenarios, where integrating expert knowledge can be highly beneficial. Additionally, improving the stability and consistency of responses to different

prompts remains a broader challenge in LLM-based systems. We recognize the significance of this issue and will continue to explore strategies to further enhance prompt stability and consistency within our framework.

9 Acknowledgements

This work is supported by the National Natural Science Foundation of China (72204087), the Shanghai Planning Office of Philosophy and Social Science Youth Project (2022ETQ001), the Chenguang Program of Shanghai Education Development Foundation and Shanghai Municipal Education Commission (23CGA28), the Shanghai Pujiang Program (23PJC030), Young Elite Scientists Sponsorship Program by CAST (YESS20240562), and the Fundamental Research Funds for the Central Universities, China. We also appreciate the constructive comments from the anonymous reviewers.

Appendix

A Prompt template

We design different prompt templates ($\mathcal{P}_g$, $\mathcal{P}_m$, $\mathcal{P}_a$) and instructions ($\mathcal{I}_{\text{generate}}$, $\mathcal{I}_{\text{merge}}$, $\mathcal{I}_{\text{assign}}$) for the three sub-tasks in our framework: label generation, label aggregation & merging, and given-label classification. Each template is carefully constructed to guide the LLMs toward producing high-quality, task-specific outputs with minimal ambiguity. Table 8 provides an overview of the prompt templates and corresponding instructions for each task, illustrating how we tailor the query to maximize the LLM’s performance.

To improve the reliability and consistency of responses, we integrate format-control instructions into the prompts. For example, we explicitly include directives such as “Please return the output in JSON format” and provide a concrete JSON structure example. This approach ensures that the LLM outputs are not only correct but also well-structured for downstream processing.

References

- [1] Neha Agarwal, Geeta Sikka, and Lalit Kumar Awasthi. 2020. Enhancing web service clustering using Length Feature Weight Method for service description document vector space representation. *Expert Systems with Applications* 161 (2020), 113682.
- [2] Charu C Aggarwal and ChengXiang Zhai. 2012. A survey of text clustering algorithms. *Mining text data* (2012), 77–128.
- [3] Peter G Anick and Shivakumar Vaithyanathan. 1997. Exploiting clustering and phrases for context-based information retrieval. In *Proceedings of the 20th annual international ACM SIGIR conference on Research and development in information retrieval*. 314–323.
- [4] Florian Beil, Martin Ester, and Xiaowei Xu. 2002. Frequent term-based text clustering. In *Proceedings of the eighth ACM SIGKDD international conference on Knowledge discovery and data mining*. 436–442.
- [5] Deyu Bo, Xiao Wang, Chuan Shi, Meiqi Zhu, Emiao Lu, and Peng Cui. 2020. Structural deep clustering network. In *Proceedings of the web conference 2020*. 1400–1410.
- [6] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [7] David Carmel, Hagai Roitman, and Naama Zwerdling. 2009. Enhancing cluster labeling using wikipedia. In *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*. 139–146.
- [8] Mathilde Caron, Piotr Bojanowski, Armand Joulin, and Matthijs Douze. 2018. Deep clustering for unsupervised learning of visual features. In *Proceedings of the European conference on computer vision (ECCV)*. 132–149.
- [9] Àngel Castellanos, Juan Cigarrán, and Ana García-Serrano. 2017. Formal concept analysis for topic detection: a clustering quality experimental analysis. *Information Systems* 66 (2017), 24–42.
- [10] Jonathan Chang, Sean Gerrish, Chong Wang, Jordan Boyd-Graber, and David Blei. 2009. Reading tea leaves: How humans interpret topic models. *Advances in neural information processing systems* 22 (2009).
- [11] Jianlong Chang, Lingfeng Wang, Gaofeng Meng, Shiming Xiang, and Chunhong Pan. 2017. Deep adaptive image clustering. In *Proceedings of the IEEE international conference on computer vision*. 5879–5887.
- [12] Douglass R Cutting, David R Karger, and Jan O Pedersen. 1993. Constant interaction-time scatter/gather browsing of very large document collections. In *Proceedings of the 16th annual international ACM SIGIR conference on Research and development in information retrieval*. 126–134.
- [13] Douglass R Cutting, David R Karger, Jan O Pedersen, and John W Tukey. 2017. Scatter/gather: A cluster-based approach to browsing large document collections. In *ACM SIGIR Forum*, Vol. 51. ACM New York, NY, USA, 148–159.
- [14] Maarten De Raedt, Frédéric Godin, Thomas Demeester, Chris Devellder, and Sinch Chatlayer. 2023. IDAS: Intent Discovery with Abstractive Summarization. In *The 5th Workshop on NLP for Conversational AI*. 71.
- [15] Dorottya Demszy, Dana Movshovitz-Attias, Jeongwoo Ko, Alan Cowen, Gaurav Nemade, and Sujith Ravi. 2020. GoEmotions: A Dataset of Fine-Grained Emotions. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 4040–4054.
- [16] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).
- [17] Sifan Ding, Min Li, Tianyi Huang, and William Zhu. 2024. Local density based on weighted K-nearest neighbors for density peaks clustering. *Knowledge-Based Systems* 305 (2024), 112609.
- [18] Martin Ester, Hans-Peter Kriegel, Jörg Sander, Xiaowei Xu, et al. 1996. A density-based algorithm for discovering clusters in large spatial databases with noise. In *kdd*, Vol. 96. 226–231.
- [19] Zexuan Fei, Haoyu Zhai, Jie Yang, Bin Wang, and Yan Ma. 2025. Discovering generalized clusters with adaptive mixture density-based clustering. *Knowledge-Based Systems* (2025), 113250.
- [20] Jack FitzGerald, Christopher Hench, Charith Peris, Scott Mackie, Kay Rottmann, Ana Sanchez, Aaron Nash, Liam Urbach, Vishesh Kakarala, Richa Singh, et al. 2023. MASSIVE: A 1M-Example Multilingual Natural Language Understanding Dataset with 51 Typologically-Diverse Languages. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 4277–4302.
- [21] Renchu Guan, Hao Zhang, Yanchun Liang, Fausto Giunchiglia, Lan Huang, and Xiaoyue Feng. 2020. Deep feature-based text clustering and its explanation. *IEEE Transactions on Knowledge and Data Engineering* 34, 8 (2020), 3669–3680.
- [22] Xifeng Guo, Xinwang Liu, En Zhu, and Jianping Yin. 2017. Deep clustering with convolutional autoencoders. In *Neural Information Processing: 24th International Conference, ICONIP 2017, Guangzhou, China, November 14–18, 2017, Proceedings, Part II* 24. Springer, 373–382.
- [23] Guoxiu He and Chen Huang. 2025. Few-shot medical relation extraction via prompt tuning enhanced pre-trained language model. *Neurocomputing* 633 (2025), 129752.
- [24] Guoxiu He, Meicong Zhang, Tiancheng Su, Li Ma, and Xiaomin Zhu. 2025. Enhancing belief consistency of Large Language Model agents in decision-making process based on attribution theory. *Expert Systems with Applications* (2025), 129273.
- [25] Peihao Huang, Yan Huang, Wei Wang, and Liang Wang. 2014. Deep embedding network for clustering. In *2014 22nd International conference on pattern recognition*. IEEE, 1532–1537.
- [26] Xin Huang, Fan Yang, Guanqiu Qi, Yuanyuan Li, Ranqiao Zhang, and Zhiqin Zhu. 2024. Deep attributed graph clustering with feature consistency contrastive and topology enhanced network. *Knowledge-Based Systems* 305 (2024), 112634.
- [27] Stephen C Johnson. 1967. Hierarchical clustering schemes. *Psychometrika* 32, 3 (1967), 241–254.
- [28] Nikitas-Rigas Kalogeropoulos, George Kontogiannis, and Christos Makris. 2025. Spectral clustering and query expansion using embeddings on the graph-based extension of the set-based information retrieval model. *Expert Systems with Applications* 263 (2025), 125771.
- [29] Harold W Kuhn. 1955. The Hungarian method for the assignment problem. *Naval research logistics quarterly* 2, 1-2 (1955), 83–97.
- [30] Sehyun Kwon, Jaeseung Park, Minkyu Kim, Jaewoong Cho, Ernest K Ryu, and Kangwook Lee. 2023. Image Clustering Conditioned on Text Criteria. *arXiv preprint arXiv:2310.18297* (2023).
- [31] Sangho Lee, Chihyeon Choi, and Youngdoo Son. 2024. Deep time-series clustering via latent representation alignment. *Knowledge-Based Systems* 303 (2024), 112434.
- [32] Haoran Li, Abhinav Arora, Shuohui Chen, Anchit Gupta, Sonal Gupta, and Yashar Mehdad. 2021. MTOP: A Comprehensive Multilingual Task-Oriented Semantic Parsing Benchmark. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*. 2950–2962.
- [33] Shu Li, Lixin Han, Yang Wang, Yonglin Pu, Jun Zhu, and Jingxian Li. 2024. Contrastive clustering based on generalized bias-variance decomposition. *Knowledge-Based Systems* 305 (2024), 112601.
- [34] Zhe Liu, Haoye Qiu, Muhammet Deveci, Witold Pedrycz, and Patrick Siarry. 2025. Multi-view neutrosophic c-means clustering algorithms. *Expert Systems with Applications* 260 (2025), 125454.
- [35] Stuart Lloyd. 1982. Least squares quantization in PCM. *IEEE transactions on information theory* 28, 2 (1982), 129–137.
- [36] Bing Ma and Hai Zhuge. 2024. Automatic construction of classification dimensions by clustering texts based on common words. *Expert Systems with Applications* 238 (2024), 122292.
- [37] Vivek Mehta, Seema Bawa, and Jasmeet Singh. 2021. Stamantic clustering: combining statistical and semantic features for clustering of large text datasets. *Expert Systems with Applications* 174 (2021), 114710.
- [38] Niklas Muennighoff, Nouamane Tazi, Loic Magne, and Nils Reimers. 2023. MTEB: Massive Text Embedding Benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*. 2014–2037.
- [39] Roberto Navigli and Giuseppe Crisafulli. 2010. Inducing word senses to improve web search result clustering. In *Proceedings of the 2010 conference on empirical methods in natural language processing*. 116–126.
- [40] OpenAI. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [41] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.
- [42] Junbiao Pang, Anjing Hu, and Qingming Huang. 2025. Bundle fragments into a whole: Mining more complete clusters via submodular selection of interesting webpages for web topic detection. *Expert Systems with Applications* 260 (2025), 125125.
- [43] Sungwon Park, Sungwon Han, Sundong Kim, Danu Kim, Sungkyu Park, Seunghoon Hong, and Meeyoung Cha. 2021. Improving unsupervised image clustering with robust learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 12278–12287.
- [44] Guo-Jun Qi, Charu C Aggarwal, and Thomas Huang. 2012. Community detection with edge content in social media networks. In *2012 IEEE 28th International conference on data engineering*. IEEE, 534–545.
- [45] Tingting Qi, Xiangchu Feng, Bian Gao, and Kun Wang. 2024. An end-to-end Graph Convolutional Network for Semi-supervised Subspace Clustering via label self-expressiveness. *Knowledge-Based Systems* 286 (2024), 111393.
- [46] Lina Ren, Yongbin Qin, Yanping Chen, Chuan Lin, and Ruizhang Huang. 2023. Deep document clustering via adaptive hybrid representation learning. *Knowledge-Based Systems* 281 (2023), 111058.
- [47] Yazhou Ren, Ni Wang, Mingxia Li, and Zenglin Xu. 2020. Deep density-based image clustering. *Knowledge-Based Systems* 197 (2020), 105841.
- [48] Frédéric Ros and Rabia Riad. 2024. DLCS: A deep learning-based Clustering solution without any clustering algorithm, Utopia? *Knowledge-Based Systems* 296 (2024), 111834.
- [49] Satu Elisa Schaeffer. 2007. Graph clustering. *Computer science review* 1, 1 (2007), 27–64.
- [50] Boheng Sheng, Jiacheng Yao, Meicong Zhang, and Guoxiu He. 2025. Dynamic Chunking and Selection for Reading Comprehension of Ultra-Long Context

- in Large Language Models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Vienna, Austria, 31857–31876. doi:10.18653/v1/2025.acl-long.1538
- [51] Xin Song, Zhikai Xue, Guoxiu He, Jiawei Liu, and Wei Lu. 2025. Interweaving Memories of a Siamese Large Language Model. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 25155–25163.
- [52] Hongjin Su, Weijia Shi, Jungo Kasai, Yizhong Wang, Yushi Hu, Mari Ostendorf, Wen-tau Yih, Noah A Smith, Luke Zettlemoyer, and Tao Yu. 2022. One embedder, any task: Instruction-finetuned text embeddings. *arXiv preprint arXiv:2212.09741* (2022).
- [53] Yaling Tao, Kentaro Takagi, and Kouta Nakata. 2021. Clustering-friendly representation learning via instance discrimination and feature decorrelation. *arXiv preprint arXiv:2106.00131* (2021).
- [54] Fei Tian, Bin Gao, Qing Cui, Enhong Chen, and Tie-Yan Liu. 2014. Learning deep representations for graph clustering. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 28.
- [55] Pucktada Treeratpituk and Jamie Callan. 2006. Automatically labeling hierarchical clusters. In *Proceedings of the 2006 international conference on Digital government research*. 167–176.
- [56] Vijay Viswanathan, Kiril Gashteovski, Kiril Gashteovski, Carolin Lawrence, Tongshuang Wu, and Graham Neubig. 2024. Large Language Models Enable Few-Shot Clustering. *Transactions of the Association for Computational Linguistics* 12 (2024), 321–333.
- [57] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533* (2022).
- [58] Xiujuan Wang, Keke Wang, Kangmiao Chen, Zhengxiang Wang, and Kangfeng Zheng. 2024. Unsupervised twitter social bot detection using deep contrastive graph clustering. *Knowledge-Based Systems* 293 (2024), 111690.
- [59] Zihan Wang, Jingbo Shang, and Ruiqi Zhong. 2023. Goal-Driven Explainable Clustering via Language Descriptions. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 10626–10649. doi:10.18653/v1/2023.emnlp-main.657
- [60] Jianlong Wu, Keyu Long, Fei Wang, Chen Qian, Cheng Li, Zhouchen Lin, and Hongbin Zha. 2019. Deep comprehensive correlation mining for image clustering. In *Proceedings of the IEEE/CVF international conference on computer vision*. 8150–8159.
- [61] Jiaming Xu, Peng Wang, Guanhua Tian, Bo Xu, Jun Zhao, Fangyuan Wang, and Hongwei Hao. 2015. Short text clustering via convolutional neural networks. In *Proceedings of the 1st Workshop on Vector Space Modeling for Natural Language Processing*. 62–69.
- [62] Beihua Yang, Peng Song, Yuanbo Cheng, Zhaowei Liu, and Yanwei Yu. 2025. Label completion based concept factorization for incomplete multi-view clustering. *Knowledge-Based Systems* 310 (2025), 112953.
- [63] Jianwei Yang, Devi Parikh, and Dhruv Batra. 2016. Joint unsupervised learning of deep representations and image clusters. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 5147–5156.
- [64] Yi Yang, Dong Xu, Feiping Nie, Shuicheng Yan, and Yueting Zhuang. 2010. Image clustering using local discriminant models and global integration. *IEEE Transactions on Image Processing* 19, 10 (2010), 2761–2773.
- [65] Jiacheng Yao, Xin Xu, and Guoxiu He. 2025. Metacognitive symbolic distillation framework for multi-choice machine reading comprehension. *Knowledge-Based Systems* 312 (2025), 113130.
- [66] Hao Yin, Austin R Benson, Jure Leskovec, and David F Gleich. 2017. Local higher-order graph clustering. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*. 555–564.
- [67] Chao Zhang, Fangbo Tao, Xiushi Chen, Jiaming Shen, Meng Jiang, Brian Sadler, Michelle Vanni, and Jiawei Han. 2018. Taxogen: Unsupervised topic taxonomy construction by adaptive term embedding and clustering. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2701–2709.
- [68] Wen Zhang, Xijin Tang, and Taketoshi Yoshida. 2015. TESC: An approach to TExt classification using Semi-supervised Clustering. *Knowledge-Based Systems* 75 (2015), 152–160.
- [69] Yujie Zhang, Yan Jiang, Pengwei Yan, Zhuoren Jiang, Chenxi Lin, Guoxiu He, and Xiaozhong Liu. 2025. Risks to Scientific Peer Review in the Era of Technoscientific Acceleration: Evidence from AI Research. *Available at SSRN 5287265* (2025).
- [70] Yuwei Zhang, Zihan Wang, and Jingbo Shang. 2023. ClusterLLM: Large Language Models as a Guide for Text Clustering. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 13903–13920.
- [71] Sheng Zhou, Hongjia Xu, Zhuonan Zheng, Jiawei Chen, Jiajun Bu, Jia Wu, Xin Wang, Wenwu Zhu, Martin Ester, et al. 2022. A comprehensive survey on deep clustering: Taxonomy, challenges, and future directions. *arXiv preprint arXiv:2206.07579* (2022).
- [72] Yang Zhou, Hong Cheng, and Jeffrey Xu Yu. 2009. Graph clustering based on structural/attribute similarities. *Proceedings of the VLDB Endowment* 2, 1 (2009), 718–729.